

Les objets distribués: Sockets

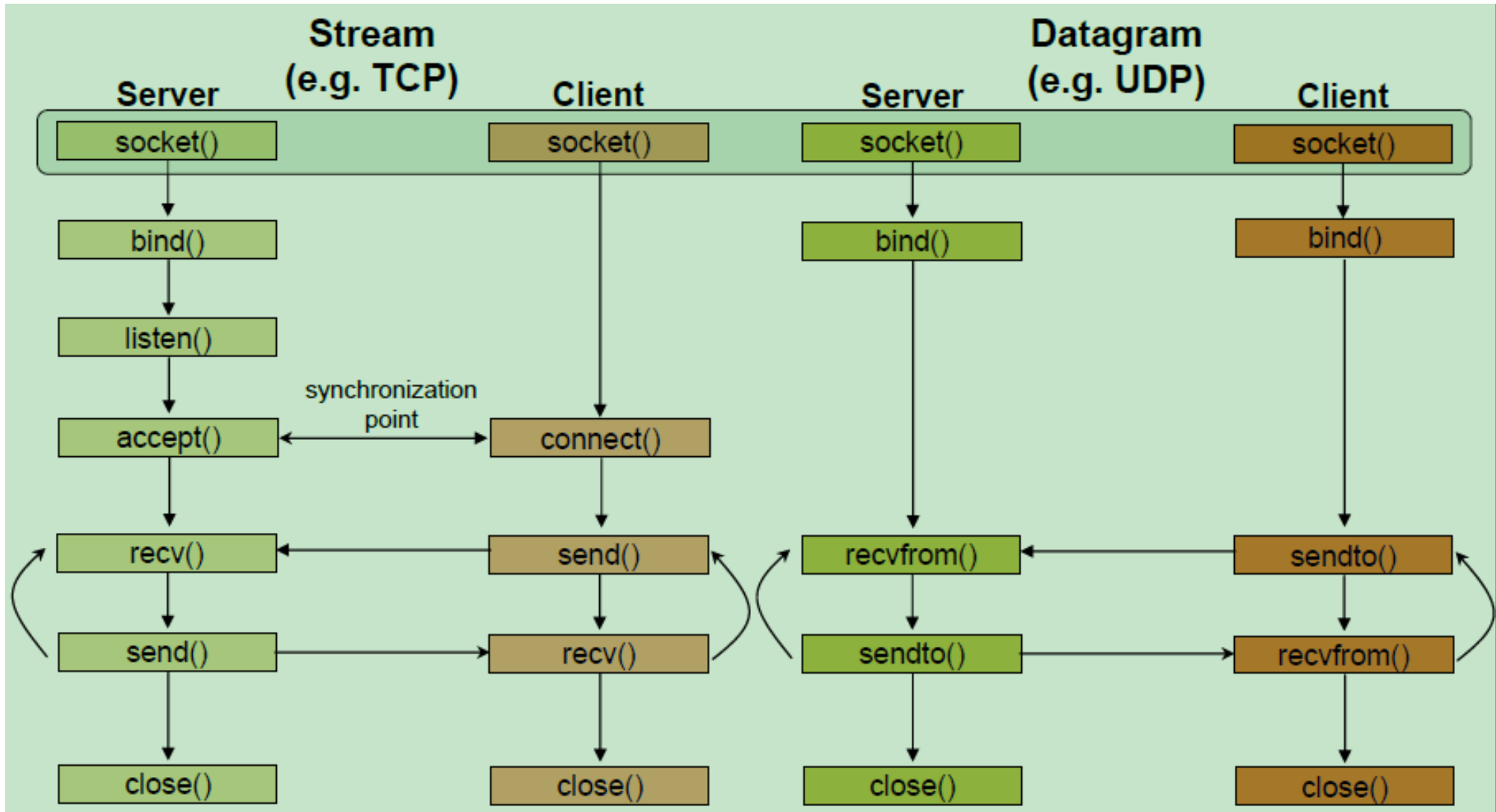
Principe des sockets

- Les sockets (ou prises) sont une abstraction logicielle qui permet la communication bidirectionnelle (deux parties peuvent envoyer et recevoir des données simultanément) entre deux programmes sur un réseau.
- Elles offrent une manière flexible et puissante pour que les programmes interagissent sur un réseau, permettant la création d'applications distribuées, de serveurs Web, de services de messagerie, et bien d'autres.
- Les sockets sont souvent utilisés dans une architecture client-serveur, où un programme agit en tant que serveur écoutant les connexions entrantes, et d'autres programmes agissent en tant que clients se connectant au serveur.
- La communication via des sockets repose généralement sur un protocole spécifique, tel que TCP (Transmission Control Protocol) ou UDP (User Datagram Protocol). TCP offre une communication fiable et orientée connexion, tandis qu'UDP offre une communication plus légère mais sans garantie de livraison des paquets.

Principe des sockets

- Deux types de sockets principaux : les sockets côté serveur et les sockets côté client. Le socket côté serveur écoute les connexions entrantes, tandis que le socket côté client est utilisé par les clients pour se connecter au serveur.
- Une fois qu'une connexion est établie entre le client et le serveur, des flux de données peuvent être créés des deux côtés. Ces flux permettent la lecture et l'écriture de données sur la connexion.
- Les sockets sont identifiés par une adresse IP et un numéro de port. L'adresse IP identifie l'hôte, et le port spécifie le service sur cet hôte. Les sockets sont liés à une adresse et un port spécifiques lors de leur création.
- Le cycle de vie d'une communication via des sockets implique généralement la création d'un socket, la connexion (pour le client), l'acceptation de connexions entrantes (pour le serveur), la communication à travers les flux, puis la fermeture de la connexion lorsque la communication est terminée.

Types des sockets



Les sockets en Java

- En Java, la gestion des sockets est facilitée par le package ***java.net***, il fournit des classes pour la création d'applications réseau, y compris la communication via des sockets.
- Les classes principales pour la gestion des sockets en Java sont:
 - ***Socket*** est utilisé par les clients pour se connecter à un serveur en mode TCP.
 - ***ServerSocket*** est utilisé par les serveurs pour écouter les connexions entrantes en TCP.
 - ***DatagramSocket*** est utilisé pour l'envoi et la réception de datagrammes en UDP.
- L'utilisation des sockets en mode UDP (User Datagram Protocol) en Java implique quelques différences par rapport à l'utilisation de sockets TCP.

Les sockets en Java – Mode TCP

- Pour créer un socket client en Java, on instancie un objet de la classe **Socket** en fournissant l'adresse IP et le port du serveur auquel on souhaite se connecter.

```
Socket clientSocket = new Socket("127.0.0.1", 8080);
```

- Pour créer un socket serveur en Java, on instancie un objet de la classe **ServerSocket** en spécifiant le port sur lequel le serveur écoutera les connexions entrantes.

```
ServerSocket serverSocket = new ServerSocket(8080);
```

- Pour accepter les connexions entrantes sur le côté serveur, on utilise la méthode **accept()** de l'objet **ServerSocket**. Cette méthode retourne un objet **Socket** représentant la connexion établie avec un client.

```
Socket clientSocket = serverSocket.accept();
```

Les sockets en Java – Mode TCP

- Une fois qu'une connexion est établie, on peut obtenir des flux de données associés au socket pour la communication. On utilise souvent ***InputStream*** et ***OutputStream*** pour lire et écrire des données.

```
InputStream inputStream = clientSocket.getInputStream();  
OutputStream outputStream = clientSocket.getOutputStream();
```

- On utilise les méthodes de lecture et d'écriture des flux pour échanger des données entre le client et le serveur.

```
// Lecture côté serveur  
BufferedReader reader = new BufferedReader(new InputStreamReader(inputStream));  
String clientMessage = reader.readLine();  
  
// Écriture côté serveur  
PrintWriter writer = new PrintWriter(outputStream, true);  
writer.println("Message du serveur : Bonjour, client !");
```

Les sockets en Java – Mode TCP

- Une fois la communication terminée, il est important de fermer les sockets pour libérer les ressources.

```
clientSocket.close();  
serverSocket.close();
```

Les sockets en Java – Mode TCP - Exemple

```
1- import java.io.*;
2  import java.net.Socket;
3
4- public class TCPClient {
5-     public static void main(String[] args) {
6         final String SERVER_IP = "127.0.0.1";
7         final int SERVER_PORT = 12345;
8
9         try {
10             // Connexion au serveur
11             Socket socket = new Socket(SERVER_IP, SERVER_PORT);
12             System.out.println("Connecté au serveur.");
13
14             // Flux de lecture et écriture pour communiquer avec le serveur
15             BufferedReader reader = new BufferedReader(new InputStreamReader(socket.getInputStream()));
16             PrintWriter writer = new PrintWriter(socket.getOutputStream(), true);
17
18             // Envoi d'un message au serveur
19             writer.println("Bonjour, serveur !");
20
21             // Lecture et affichage de la réponse du serveur
22             String serverResponse = reader.readLine();
23             System.out.println("Réponse du serveur : " + serverResponse);
24
25             // Fermeture des flux et du socket
26             reader.close();
27             writer.close();
28             socket.close();
29-         } catch (IOException e) {
30             e.printStackTrace();
31         }
32     }
33 }
```

```
1- import java.io.*;
2  import java.net.ServerSocket;
3  import java.net.Socket;
4- public class TCPServer {
5-     public static void main(String[] args) {
6         final int PORT = 12345;
7         try {
8             ServerSocket serverSocket = new ServerSocket(PORT);
9             System.out.println("Serveur en attente de connexions...");
10
11             // Attente de connexion d'un client
12             Socket clientSocket = serverSocket.accept();
13             System.out.println("Client connecté depuis : " + clientSocket.getInetAddress());
14
15             // Flux de lecture et écriture pour communiquer avec le client
16             BufferedReader reader = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));
17             PrintWriter writer = new PrintWriter(clientSocket.getOutputStream(), true);
18
19             // Lecture et affichage du message du client
20             String clientMessage = reader.readLine();
21             System.out.println("Message du client : " + clientMessage);
22
23             // Réponse au client
24             writer.println("Message reçu, merci !");
25
26             // Fermeture des flux et du socket
27             reader.close(); writer.close();
28             clientSocket.close(); serverSocket.close();
29-         } catch (IOException e) {
30             e.printStackTrace();
31         }
32     }
33 }
```

Les sockets en Java – Mode UDP (Côté client)

- ***DatagramSocket*** est utilisé pour créer un socket qui sera utilisé pour envoyer des datagrammes (paquets) UDP au serveur.

```
DatagramSocket socket = new DatagramSocket();
```

- Les données à envoyer doivent d'abord être transférées en bytes avant d'être encapsulées par le ***DatagramSocket***.

```
byte[] data = message.getBytes();  
InetAddress serverAddress = InetAddress.getByName(ip);  
DatagramPacket packet = new DatagramPacket(data, data.length, serverAddress, port);
```

- Pour envoyer un datagramme, on utilise la méthode ***send()*** de l'objet ***DatagramSocket*** en fournissant le ***DatagramPacket*** à envoyer.

```
socket.send(packet);
```

- Comme avec les sockets TCP, il est important de fermer le DatagramSocket après utilisation.

```
socket.close();
```

Les sockets en Java – Mode UDP (Côté server)

- ***DatagramSocket*** est utilisé pour créer un socket qui sera utilisé pour recevoir des datagrammes (paquets) UDP du client. On doit alors spécifier le port.

```
DatagramSocket socket = new DatagramSocket(port);
```

- Pour recevoir un datagramme, on crée un ***DatagramPacket*** vide pour recevoir les données, puis on utilise la méthode ***receive()*** de l'objet ***DatagramSocket***.

```
byte[] data = new byte[1024];  
DatagramPacket packet = new DatagramPacket(data, data.length);  
socket.receive(packet);
```

- Une fois le datagramme reçu, on peut extraire les données au ***DatagramPacket*** de manière similaire à la lecture/écriture avec des flux en TCP.

```
String message = new String(packet.getData(), 0, packet.getLength());
```

- Comme avec les sockets TCP, il est important de fermer le `DatagramSocket` après utilisation.

```
socket.close();
```

Les sockets en Java – Mode UDP - Exemple

```
1- import java.net.DatagramPacket;
2- import java.net.DatagramSocket;
3- import java.net.InetAddress;
4
5- public class UDPClient {
6-     public static void main(String[] args) {
7-         try {
8-             // Create a DatagramSocket for the client
9-             DatagramSocket datagramSocket = new DatagramSocket();
10
11-             // Define the server's address and port
12-             InetAddress serverAddress = InetAddress.getByName("localhost");
13-             int serverPort = 8080;
14
15-             // Create a message to send
16-             String message = "Hello, Server!";
17
18-             // Convert the message to a byte array
19-             byte[] data = message.getBytes();
20
21-             // Create a DatagramPacket to send data to the server
22-             DatagramPacket packet = new DatagramPacket(data, data.length, serverAddress, serverPort);
23
24-             // Send the packet to the server
25-             datagramSocket.send(packet);
26
27-             // Close the DatagramSocket
28-             datagramSocket.close();
29-         } catch (Exception e) {
30-             e.printStackTrace();
31-         }
32-     }
33- }
```

```
1- import java.net.DatagramPacket;
2- import java.net.DatagramSocket;
3
4- public class UDPServer {
5-     public static void main(String[] args) {
6-         try {
7-             // Create a DatagramSocket to listen on a specific port (e.g., 8080)
8-             DatagramSocket datagramSocket = new DatagramSocket(8080);
9
10-             // Create a byte array to store received data
11-             byte[] buffer = new byte[1024];
12
13-             // Create a DatagramPacket to receive data
14-             DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
15
16-             // Receive data from the client
17-             datagramSocket.receive(packet);
18
19-             // Extract and display the received message
20-             String message = new String(packet.getData(), 0, packet.getLength());
21-             System.out.println("Received from client: " + message);
22
23-             // Close the DatagramSocket
24-             datagramSocket.close();
25-         } catch (Exception e) {
26-             e.printStackTrace();
27-         }
28-     }
29- }
```